

Migrations Overview?

Now that we have a database and we have connected to it through our application, we need to be able to make changes to our database. This is where migrations come in. Migrations are version-controlled files used to manage database schema changes. They allow you to create, modify or delete tables and columns in a structured and repeatable way.

I've been working with PHP and SQL databases for a long time and I remember when I was working with the Codeigniter framework, we would have to map out our database schema with all of the tables, columns and fields. We would do this through something like PHPMyAdmin or even through the command line. This was a lot of work and it was a lot of manual labor. Now we can just create a migration file and run it through the command line and it will create the table for us. This is a huge time saver and it makes our lives a lot easier.

Default Migrations

Laravel comes with a few default migrations that are used to create the tables that are used by the framework. These migrations are located in the `database/migrations` directory. Go ahead and open up that directory.

The files that you see may differ depending on the version of Laravel that you are using. I am using Laravel 11 and I see the following 3 files:

- `0001_01_01_000000_create_users_table.php`
- `0001_01_01_000001_create_cache_table.php`
- `0001_01_01_000002_create_jobs_table.php`

These files are used to create the tables that are needed for users/authentication, caching and background jobs or tasks. You can see that the file names are prefixed with a timestamp. This is to ensure that the files are run in the correct order. The timestamp is in the format of `YYYY_MM_DD_HHMMSS`. This is the order that the files will be run in.

We have an issue here and that is that there is a default 'jobs' table. This table is used by Laravel to queue jobs, meaning tasks. So since we are creating a job listing website, we can't call our table 'jobs'. We will need to change the name of the table to something else. I am going to use the name 'job_listings' instead.

We will create our own migrations in a little bit. Right now, I want to show you how to run the migrations that are already in the `database/migrations` directory. Before we do that, let's look at the code in the `0001_01_01_000000_create_users_table.php` file.

You will notice that there is an `up()` and a `down()` method. The `up()` method is used to create the table and the `down()` method is used to drop the table. Inside the `up()` method, you will see the `Schema` facade and `create` method is being used to create a few tables relating to users and authentication. There will be a `users`, `password_reset_tokens`, and `sessions` table. The `down()` method is used to drop the tables.

The `create` method takes a name and a closure. The closure is used to define the columns and the data types of the table.

Let's take a closer look at the code.

```
Schema::create('users', function (Blueprint $table) {  
    $table->id();  
    $table->string('name');  
    $table->string('email')->unique();  
    $table->timestamp('email_verified_at')->nullable();  
    $table->string('password');  
    $table->rememberToken();  
    $table->timestamps();  
});
```

Here it is creating a `users` table with the following columns:

- `id` - auto-incrementing integer
- `name` - string
- `email` - string
- `email_verified_at` - timestamp
- `password` - string
- `remember_token` - string
- `created_at` - timestamp
- `updated_at` - timestamp

The `id` column is the primary key and it is an auto-incrementing integer.

The `remember_token` column is used to store the token that is used to remember the user when they come back to the site. This is used by the `remember me` feature. The `remember_token` column is not created by default, we have to add it manually.

The `timestamps` method is used to create the `created_at` and `updated_at` columns. These columns are used to store the date and time that a record was created and updated.

If you wanted your users to have different or additional columns, you would add them to the closure. For example, if you wanted to add a `phone` column, you would add the following code to the closure:

```
$table->string('phone');
```

Or maybe instead of a name column, you wanted to add a `first_name` and `last_name` column, you would add the following code to the closure:

```
$table->string('first_name');  
$table->string('last_name');
```

I am just going to keep the defaults though.

It's important to mention that if you do edit anything here or anywhere else, you need to run the `php artisan migrate` command to update the database. Which we will do in a minute.

Let's look at the next block of code.

```
Schema::create('password_reset_tokens', function (Blueprint $table) {  
    $table->string('email')->primary();  
    $table->string('token');  
    $table->timestamp('created_at')->nullable();  
});
```

Here it is creating a `password_reset_tokens` table with the following columns:

- `email` - string
- `token` - string
- `created_at` - timestamp

The `email` column is the primary key and it is used to store the email address of the user. The `token` column is used to store the token that is used to reset the user's password. The `created_at` column is used to store the date and time that the token was created.

Let's look at the last block of code.

```
Schema::create('sessions', function (Blueprint $table) {  
    $table->string('id')->primary();  
    $table->foreignId('user_id')->nullable()->index();  
    $table->string('ip_address', 45)->nullable();  
    $table->text('user_agent')->nullable();  
    $table->longText('payload');  
    $table->integer('last_activity')->index();  
});
```

Here it is creating a `sessions` table with the following columns:

- `id` - string
- `user_id` - integer
- `ip_address` - string
- `user_agent` - text
- `payload` - long text
- `last_activity` - integer

The `id` column is a string and is the primary key and it is used to store the session id. The `user_id` column is used to store the id of the user that is logged in. The `ip_address` column is used to store the ip address of the user. The `user_agent` column is used to store the user agent of the user. The `payload` column is used to store the session data. The `last_activity` column is used to store the date and time of the last activity.

Now that we have looked at the default migrations, let's run them.

Running Migrations

To run the migrations, we need to use the `migrate` command. Open up your terminal and run the following command:

```
php artisan migrate
```

This will run all of the migrations that are in the `database/migrations` directory.

Now you can check your database and you should have all of those tables. You can check through PG Admin by going to the database and then click on `Schemas` and then click on `public` and then click on `Tables`. You should see all of the tables that were created. You can also check within the psql shell or in Tinker.

There are some other handy commands that you can use with migrations. Let's see them by running the following command:

```
php artisan migrate:help
```

You will see the following:

```
└ migrate
  └ migrate:fresh
  └ migrate:install
  └ migrate:refresh
  └ migrate:reset
  └ migrate:rollback
  └ migrate:status
```

Let's look at what each of these commands do:

- `migrate` - Runs all of the migrations that are in the `database/migrations` directory.
- `migrate:fresh` - Completely drops all tables and re-runs all migrations. Useful for starting with a clean slate.
- `migrate:install` - Creates the migrations table.
- `migrate:refresh` - Rolls back all migrations and then re-applies them. Useful for resetting migrations without dropping the entire database schema.
- `migrate:reset` - Rolls back all of the migrations that have been run.
- `migrate:rollback` - Rolls back the last migration that was run. This is useful if you make a mistake and need to undo it.
- `migrate:status` - Shows the status of the migrations.

Using commands like `migrate:fresh` and `migrate:reset` are usually only used in development. In production, you would never want to use these commands. You would lose all of your data.